

Assignment No. 08

- **TITLE** – Use a singly linked list to represent and perform addition of polynomials, enabling dynamic storage and easy insertion of terms. Extend this to a Generalized Linked List (GLL) to handle sparse polynomial representations efficiently.

➤ **THEORY** –

Polynomials are mathematical expressions with variables and coefficients. Representing them efficiently in computer memory is crucial for mathematical computations. Linked lists provide dynamic storage suitable for polynomial operations.

SINGLY LINKED LIST FOR POLYNOMIALS

Structure

Each node contains:

Coefficient (coeff)

Exponent (exp)

Pointer to next term (next)

Advantages

Dynamic memory allocation

Easy insertion and deletion

Efficient for sparse polynomials

No memory wastage

Operations

Insertion: Terms inserted in descending exponent order

Addition: Merge two sorted polynomial lists

Traversal: Display polynomial terms

GENERALIZED LINKED LIST (GLL) FOR SPARSE POLYNOMIALS

Structure

GLL node contains:

Flag (0 for data, 1 for down pointer)

Union of:

Data (coeff, exp)

Down pointer to sub-polynomial

Next pointer

Sparse Polynomial Representation

Sparse polynomials have few non-zero terms compared to their highest degree. GLL efficiently represents them by storing only non-zero terms.

Benefits of GLL for Sparse Polynomials

- Memory efficient for large exponents
- Handles hierarchical polynomial structures
- Flexible representation for complex expressions

- Optimal storage utilization

ALGORITHMS

Polynomial Addition Algorithm

- Initialize result polynomial as empty
- Traverse both polynomials simultaneously
- Compare exponents of current terms
- If exponents equal, add coefficients and insert
- If exponents unequal, insert term with higher exponent
- Copy remaining terms from longer polynomial
- Return result polynomial

Time Complexity: $O(m+n)$ where m, n are term counts

Space Complexity: $O(m+n)$ for result storage

APPLICATIONS

1. Computer Algebra Systems
2. Mathematical Computing
3. Signal Processing
4. Scientific Computations
5. Graph Theory Algorithms

PERFORMANCE ANALYSIS

1. Linked List vs Array

Arrays: Fixed size, memory wastage for sparse

Linked Lists: Dynamic, efficient for sparse data

2. Memory Efficiency

Stores only non-zero terms

No pre-allocation required

Grows as needed

➤ CODE –

1. poly_add_ll.cpp

```
#include <iostream>
using namespace std;
```

```
struct Node{
    int coeff;
    int exp;
    Node* next;
};
```

```
void insertNode(Node*& head, int coeff, int exp) {
    Node* newNode = new Node();
    newNode->coeff = coeff;
```

```

newNode->exp = exp;
newNode->next = nullptr;

if (head == nullptr || head->exp < exp) {
    newNode->next = head;
    head = newNode;
} else {
    Node* temp = head;
    while (temp->next != nullptr && temp->next->exp >= exp) {
        if (temp->next->exp == exp) {
            temp->next->coeff += coeff;
            delete newNode;
            return;
        }
        temp = temp->next;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}

}

void printPolynomial(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->coeff << "x^" << temp->exp;
        temp = temp->next;
        if (temp != nullptr) cout << " + ";
    }
    cout << endl;
}

Node* addPolynomials(Node* poly1, Node* poly2) {
    Node* result = nullptr;
    Node* p1 = poly1;
    Node* p2 = poly2;

    while (p1 != nullptr && p2 != nullptr) {
        if (p1->exp > p2->exp) {
            insertNode(result, p1->coeff, p1->exp);
            p1 = p1->next;
        } else if (p1->exp < p2->exp) {
            insertNode(result, p2->coeff, p2->exp);
            p2 = p2->next;
        } else {
            insertNode(result, p1->coeff + p2->coeff, p1->exp);
            p1 = p1->next;
            p2 = p2->next;
        }
    }

    while (p1 != nullptr) {
        insertNode(result, p1->coeff, p1->exp);
    }
}

```

```

        p1 = p1->next;
    }

    while (p2 != nullptr) {
        insertNode(result, p2->coeff, p2->exp);
        p2 = p2->next;
    }

    return result;
}

int main(){
    Node* poly1 = nullptr;
    Node* poly2 = nullptr;

    insertNode(poly1, 5, 3);
    insertNode(poly1, 4, 2);
    insertNode(poly1, 2, 0);

    insertNode(poly2, 3, 3);
    insertNode(poly2, 2, 1);
    insertNode(poly2, 1, 0);

    cout << "First Polynomial: ";
    printPolynomial(poly1);

    cout << "Second Polynomial: ";
    printPolynomial(poly2);

    Node* result = addPolynomials(poly1, poly2);
    cout << "Resultant Polynomial after Addition: ";
    printPolynomial(result);
    return 0;
}

```

2. gen_ll.cpp

```

#include <iostream>
using namespace std;

struct GLLNode {
    bool isAtom;
    union {
        struct {
            int row;
            int col;
            int val;
        } data;
        GLLNode* sublist;
    };
    GLLNode* next;
};

```

```

GLLNode* createAtom(int r, int c, int v) {
    GLLNode* node = new GLLNode;
    node->isAtom = true;
    node->data.row = r;
    node->data.col = c;
    node->data.val = v;
    node->next = nullptr;
    return node;
}

GLLNode* createSublist(GLLNode* list) {
    GLLNode* node = new GLLNode;
    node->isAtom = false;
    node->sublist = list;
    node->next = nullptr;
    return node;
}

void appendNode(GLLNode*& head, GLLNode* node) {
    if (!head) head = node;
    else {
        GLLNode* temp = head;
        while(temp->next) temp = temp->next;
        temp->next = node;
    }
}

void displayGLL(GLLNode* head) {
    GLLNode* temp = head;
    while(temp) {
        if(temp->isAtom) {
            cout << "(" << temp->data.row << "," << temp->data.col << ")=" << temp->data.val;
        } else {
            cout << "{ ";
            displayGLL(temp->sublist);
            cout << " }";
        }
        if(temp->next) cout << " -> ";
        temp = temp->next;
    }
}

int main() {
    GLLNode* matrix = nullptr;

    GLLNode* row0 = nullptr;
    appendNode(row0, createAtom(0,1,5));
    appendNode(row0, createAtom(0,3,8));
    appendNode(matrix, createSublist(row0));

    GLLNode* row1 = nullptr;

```

```

appendNode(row1, createAtom(1,0,3));
appendNode(row1, createAtom(1,2,6));
appendNode(matrix, createSublist(row1));

cout << "Sparse matrix GLL representation:\n";
displayGLL(matrix);
cout << endl;

return 0;
}

```

➤ OUTPUT –

1. poly_add_ll.cpp

First Polynomial: $5x^3 + 4x^2 + 2x^0$

Second Polynomial: $3x^3 + 2x^1 + 1x^0$

Resultant Polynomial after Addition: $8x^3 + 4x^2 + 2x^1 + 3x^0$

2. gen_ll.cpp

Sparse matrix GLL representation:

{ (0,1)=5 -> (0,3)=8 } -> { (1,0)=3 -> (1,2)=6 }

➤ CONCLUSION –

This lab assignment implementation demonstrates efficient polynomial handling using linked list structures, providing optimal solutions for mathematical computations in programming.